COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/26	**DATE SUBMITTED:**	10/16/26
NAME:	Arididdon, Gabriel Dwyane D. Esguerra, Paula G.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

LO A & ILO B:

Part 1: Simple One-Threaded Example

Output

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

The join() command waited for the printing of integers and printing the string to finish first before executing the main. Because without using join function, the main thread would finish the task early, the function of t1 will not execute, resulting it wouldn't print at all, resulting in an error.

Part 2: Multi-Threaded Example

Output

```
1 : Competent2 : Computer3 : Engineers0 : T.I.P.
```

Analysis

Based on the output, the threads executed concurrently that is why they are in the same line and overlappingly. The outcome of the code will execute all at the same time, it can be in different places, and different set, if they are printed multiple times. The execution time is faster than the regular execution time if multi-threaded isn't used.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions.

Questions

1. **Write a definition of multithreading and its advantages/disadvantages.**

   Aridiodon

   One advantage is improved responsiveness, allowing applications to continue working while other tasks run in the background. It also improves CPU utilization and may increase performance by allowing tasks to run in parallel. However, multithreading can make programs more complex and may cause synchronization issues, race conditions, or deadlocks when threads compete for shared resources.

   Esguerra

   The main function of multi-threading is to simultaneously run or execute multiple tasks. The common advantages of multi-threading is it can enhance performance by its execution time, it can also be better use of cache storage in utilization of resources and cpu resources. While the common disadvantages are it can have complex debugging and testing processes, overhead switching of context, and unpredictable results.

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**

   Aridiodon

   Multithreading is useful in applications that need to handle multiple tasks efficiently. In web browsers, it allows activities such as loading pages, playing videos, and responding to user input to happen concurrently. In online games, separate threads can handle sound, user input, and background processing to improve responsiveness. In web servers, multithreading allows multiple client requests to be handled concurrently, improving the application's ability to serve multiple users.

   Esguerra

   Multi-threading increases responsiveness in interactive applications, if one thread is blockers, other threads can still continue executing. Second, threads share the memory and resources of their parent process by default, like message passing and shared memory. And third, creating and managing threads, we can avoid the need to separate allocations.

3. **Differentiate between parallelism and concurrency**

   Aridiodon

   Concurrency is the ability to manage multiple tasks that make progress during overlapping periods, even if they take turns executing on a single CPU core. Parallelism, on the other hand, involves executing multiple tasks simultaneously, typically using multiple CPU cores. The main goal of concurrency is to improve responsiveness and manage multiple tasks efficiently, while parallelism aims to increase processing speed. For example, switching between downloading a file and responding to

user input demonstrates concurrency, while processing different parts of an image simultaneously on separate CPU cores demonstrates parallelism.

Esguerra
Concurrency can manage multiple tasks to make processes during overlapping time periods. They can be run simultaneously on multiple processors. Parallelism can execute multiple tasks simultaneously using multiple processing units. It improves during the work across the processor's throughout and its computational speed in dividing work.

Part B: Create C++ Code and show a solution that satisfies the given requirements below

Code

```
#include <chrono>
#include <iostream>
#include <mutex>
#include <thread>

int total = 0;
std::mutex mtx;

void add(int n)
{

    std::this_thread::sleep_for(std::chrono::milliseconds(n * 10));
    std::lock_guard<std::mutex> lock(mtx);
    total += n;
}

int show()
{
    std::lock_guard<std::mutex> lock(mtx);
    return total;
}

int main()
{
    std::thread t1(add, 10);
```

```
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    std::cout << "Display             : " << show() << std::endl;
    t1.join();
    std::cout << "After T1.join()     : " << show() << std::endl;
    t2.join();
    std::cout << "After T2.join()     : " << show() << std::endl;
    t3.join();
    std::cout << "After T3.join()     : " << show() << std::endl;
    return 0;
}
```

Output

```
Initial check (Threads running): 0
After T1.join(): 10
After T2.join(): 30
After T3.join(): 60
Final value (All joins complete): 60
```

Analysis

Aridiodon	**Esguerra**
join() makes the main thread wait for a specific thread to finish. The displayed values are 0, 10, 30, and 60, depending on which threads have completed. The final total is always 60, but intermediate values may vary depending on the execution order. A delay was placed inside add() for simulation.	This code uses three separate threads, theses threads are going to be modified and simulitously be read. The core functions are the sleep timer, and lock and add, sleep is to delay the thread so the output would be different from each other. Lock is to make it safe from changing, that would be fine by now.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Code

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <mutex>

std::mutex outputMutex;

// Function to combine two sorted halves
void merge(std::vector<int>& arr, int left, int mid, int right) {
    std::vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= right)
        temp[k++] = arr[j++];

    for (i = left, k = 0; i <= right; ++i, ++k) {
        arr[i] = temp[k];
    }
}

// Sequential Merge Sort
void sequentialMergeSort(std::vector<int>& arr, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    sequentialMergeSort(arr, left, mid);
```

```
    sequentialMergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}

// Parallel Merge Sort using threads and lambda functions
void parallelMergeSort(std::vector<int>& arr, int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    // Thread 1: Sort the left half
    std::thread leftThread([&]() {
        sequentialMergeSort(arr, left, mid);

        std::lock_guard<std::mutex> lock(outputMutex);
        std::cout << "[Thread 1] Left half sorted: ";

        for (int i = left; i <= mid; i++) {
            std::cout << arr[i] << " ";
        }

        std::cout << "\n";
    });

    // Thread 2: Sort the right half
    std::thread rightThread([&]() {
        sequentialMergeSort(arr, mid + 1, right);

        std::lock_guard<std::mutex> lock(outputMutex);
        std::cout << "[Thread 2] Right half sorted: ";

        for (int i = mid + 1; i <= right; i++) {
            std::cout << arr[i] << " ";
        }

        std::cout << "\n";
    });

    // Wait for both threads to finish
    leftThread.join();
    rightThread.join();
```

```
    // Merge the two sorted halves
    merge(arr, left, mid, right);
}

int main() {
    std::vector<int> data = {
        38, 27, 43, 3, 9, 82,
        10, 19, 50, 4, 1, 99
    };

    std::cout << "Original Array: ";
    for (int x : data) {
        std::cout << x << " ";
    }
    std::cout << "\n\n";

    // Start parallel sorting
    parallelMergeSort(data, 0, data.size() - 1);

    // Display final sorted array
    std::cout << "\nSorted Array: ";
    for (int x : data) {
        std::cout << x << " ";
    }

    std::cout << "\n\n[SUCCESS] Sorting Complete!\n";

    return 0;
}
```

Output

```
Original Array: 38 27 43 3 9 82 10 19 50 4 1 99

[Thread 1] Left half sorted: 3 9 27 38 43 82
[Thread 2] Right half sorted: 1 4 10 19 50 99

Sorted Array: 1 3 4 9 10 19 27 38 43 50 82 99

[SUCCESS] Sorting Complete!
```

Analysis

Arididdon	**Esguerra**
The program uses Parallel Merge Sort, a sorting algorithm that divides an array into two halves and sorts them using two separate threads. Each thread is responsible for sorting its assigned half, allowing both halves to be processed concurrently. Thread 1 sorts the left of the array, while Thread 2 sorts the right half. After both threads complete their tasks, the merge() function combines the two sorted halves into one fully sorted array.	Using merge function, it take two sorted adjacent part of the array, this copies them to a vector for sorting then copies them back to the original array. And parallelism, instead of sorting in halves in one thread, this code creates two working thread, it runs the lamba function, once it is sorted, it'll lock the console. If the both workin thread finishes, it'll combine to one sorted array.

Conclusion

Arididdon	**Esguerra**
I learned that concurrency lets a program overlap tasks, and that in C++ this is done by creating std::thread objects that are given a function and its arguments. join() makes the main thread wait for a thread to finish. I also learned the difference between concurrency (overlapping tasks that may share data) and parallelism (the same work running at the same time on different data), and that threads share memory, which makes data races possible and means synchronization such as std::mutex is needed.	In this hands-on-activity, learning about parallel algo and multithreading is interesting where it is possible to use sorting techniques and have immediate execution time. Learning when to use them because of the pros and cons may actually be helpful, because having to understand, is better than using it blindly. Multithreading is good, but it's randomly and unpredictable. Parallelism is not complex as it is, maybe twik will do, but that would be far from the future. So in this activity, I learned when and howmit should be use.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explaining concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used Gemini to explain hard concepts and also with some code assistance in supplementary activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.